MOHAN BABU UNIVERSITY

Anveshana Club

Implementation Plan & Roadmap

DOCUMENT VERSION: 1.0
ROLE: SENIOR TECHNICAL LEAD
STACK: MERN + CLOUDINARY

1. Source of Truth & Project Context

This Implementation Plan is the final directive for engineering execution. It strictly adheres to the following documents in order of authority:

1. **Product Requirements Document (PRD)**
2. **Technical Requirements Document (TRD)**
3. **Application Flow Document**
4. **UI/UX Design Brief**
5. **Backend Schema and API Specification**

> **Conflict Resolution Protocol:** If an implementation detail conflicts between these documents, the higher-ranking document takes precedence. The AI Coding Agent or developer must explicitly log the conflict and the resolution applied.

2. AI Coding Agent Instructions

To ensure a reliable, compliant workflow, the coding agent MUST adhere to these explicit directives:

- **Read All Specs:** Ingest the PRD, TRD, App Flow, UI/UX, and Backend Schema before writing code.
- **Strict Adherence:** Treat approved requirements and architecture decisions as the absolute source of truth.
- **Phased Execution:** Work phase by phase. Do not generate the entire application in one giant step. Wait for human validation or test passing between phases.
- **Inspect First:** Inspect the existing repository state and file contents before modifying or overwriting files.
- **Preserve Working Code:** Avoid replacing or refactoring working code unnecessarily.
- **Maintain Contracts:** Keep frontend API calls, backend routes, and database schema contracts synchronized.
- **Zero Hardcoded Secrets:** Never hardcode credentials, database URIs, JWT secrets, or Cloudinary keys. Use `process.env`.
- **Real Integration:** Implement real API and database integration immediately. Do not build mock functionality.
- **Test-Driven Progression:** Test each feature locally before proceeding to dependent features.
- **Phase Reporting:** Report changed files, completed tasks, test results, and unresolved issues after completing each phase.

3. Phased Implementation Roadmap

Phase 1: Project Setup

Objective	Initialize the monorepo structure, build tools, and baseline configuration.
Tasks	**TASK-101:** Initialize Git repository and `.gitignore`. **TASK-102:** Scaffold Frontend (`npm create vite@latest frontend -- --template react-ts`). **TASK-103:** Scaffold Backend (`npm init -y` inside `/backend`), install Express. **TASK-104:** Setup ESLint and Prettier across workspaces. **TASK-105:** Create `.env.example` files.
Dependencies	None.
Deliverables	Bootstrappable `/frontend` and `/backend` directories.
Acceptance Criteria	Running `npm run dev` starts the local servers cleanly.

Phase 2: Data Model and Backend Foundation

Objective	Establish database connectivity and define Mongoose schemas.
Tasks	**TASK-201:** Configure MongoDB Atlas connection in Express. **TASK-202:** Implement Mongoose schemas (Admin, Events, EventGalleries, TeamCategories, TeamMembers, Feedback, ClubSettings). **TASK-203:** Implement central error handling middleware. **TASK-204:** Create `/api/health` check endpoint. **TASK-205:** Write `seed.js` script to generate a test Admin and sample ClubSettings.
Dependencies	Phase 1
Deliverables	Database connection module, schema files, seed script.
Acceptance Criteria	Database connects successfully. Seed script populates DB without validation errors.
Testing	Unit test schema validation (e.g., missing required fields throw errors).

Phase 3: Admin Authentication and Security

Objective	Secure the backend and establish the Admin authentication lifecycle.
Tasks	**TASK-301:** Implement `/api/admin/auth/login` and `/logout` endpoints. **TASK-302:** Implement JWT generation and HttpOnly cookie assignment. **TASK-303:** Create `requireAuth` middleware for protected routes. **TASK-304:** Configure Helmet, CORS, and Rate Limiting. **TASK-305:** Build Frontend `/admin` login view and Auth Context wrapper.
Dependencies	Phase 2
Deliverables	Auth controllers, security middleware, React Auth Provider, Login UI.
Acceptance Criteria	Admin can log in, receive a cookie, and access a protected test route. Unauthenticated requests are rejected (401).
Testing	API testing via Postman/Jest for login success, failure, and token expiration.

Phase 4: Public Website Foundation

Objective	Scaffold the public-facing UI and design system.
Tasks	**TASK-401:** Install Tailwind CSS and configure Design Tokens (colors, fonts). **TASK-402:** Implement React Router for public routes (`/` , `/events` , `/team` , `/feedback`). **TASK-403:** Build responsive Header (Navbar) and Footer components. **TASK-404:** Build the Homepage Hero component using the poster visual reference. **TASK-405:** Create global UI components (Buttons, Loaders, Error Boundaries).
Dependencies	Phase 1
Deliverables	Tailwind config, routing tree, base layout, core UI components.
Acceptance Criteria	Public routes are navigable, responsive, and visually align with the token spec.

Phase 5: Events and Registration

Objective	End-to-end implementation of Event lifecycle.
Tasks	**TASK-501:** Backend CRUD API for Events (Admin) and list/detail API (Public). **TASK-502:** Integrate Cloudinary Node SDK for server-side banner uploads. **TASK-503:** Admin UI: Event list table, Create/Edit form. **TASK-504:** Public UI: Event listing page with Status badges. **TASK-505:** Public UI: Event Details page with registration CTA logic.
Dependencies	Phase 3, Phase 4
Deliverables	Event controllers, Cloudinary service, Admin Event views, Public Event views.
Acceptance Criteria	Admins can create/publish events. Users can view them and click the Google Form link if registration is open.
Testing	Test banner image upload formats/sizes. Test registration button states.

Phase 6: Event Galleries

Objective	Image gallery functionality for completed events.
Tasks	**TASK-601:** Backend API for uploading, ordering, and deleting array of images. **TASK-602:** Admin UI: Drag-and-drop gallery uploader for a specific event. **TASK-603:** Public UI: Responsive image grid on Event Details page. **TASK-604:** Public UI: Image lightbox/modal for full-screen viewing.
Dependencies	Phase 5
Deliverables	Gallery API, Admin gallery manager, Public gallery grid.
Acceptance Criteria	Admins can attach multiple photos to an event. Photos display in order.
Testing	Test bulk uploads, test Cloudinary deletion on image removal.

Phase 7: Core Team Management

Objective	Team directory management and display.
Tasks	**TASK-701:** Backend API for TeamCategories and TeamMembers. **TASK-702:** Admin UI: Manage Categories (Create/Order). **TASK-703:** Admin UI: Manage Members (Upload photo, assign category, links). **TASK-704:** Public UI: `/team` page with expandable category accordions and member cards.
Dependencies	Phase 3, Phase 4
Deliverables	Team CRUD APIs, Admin Team dashboard, Public Team directory.
Acceptance Criteria	Team is grouped by category. Accordions work. Social links open properly.
Testing	Verify category deletion prevents if members are attached (constraint check).

Phase 8: Feedback and Club Information

Objective	Public feedback collection and dynamic global settings.
Tasks	**TASK-801:** Backend API for Feedback submission (with rate limit) and Admin review. **TASK-802:** Backend API for updating `ClubSettings`. **TASK-803:** Public UI: `/feedback` page with 5-star selector and text form. **TASK-804:** Admin UI: Feedback inbox (View, mark reviewed). **TASK-805:** Admin UI: Settings page (Update contact info, mission statement).
Dependencies	Phase 3, Phase 4
Deliverables	Feedback API, Settings API, Public forms, Admin moderation tools.
Acceptance Criteria	Users can submit feedback. Admins can moderate it. Contact details pull from DB.
Testing	Test spam prevention (verify 429 Too Many Requests response).

Phase 9: UI Polish and Accessibility

Objective	Finalize visual fidelity, interactions, and WCAG compliance.
Tasks	**TASK-901:** Implement Skeleton loaders for all data-fetching views. **TASK-902:** Standardize success/error toast notifications. **TASK-903:** Perform keyboard navigation audit. **TASK-904:** Optimize images (`loading="lazy"` , Cloudinary params). **TASK-905:** Final visual alignment check against Anveshana sunset poster.
Dependencies	Phases 4-8
Deliverables	Polished UI components, accessibility fixes, optimized assets.
Acceptance Criteria	Lighthouse Accessibility score > 95. No UI layout shifts (CLS) on load.
Testing	Screen reader tests, automated Lighthouse audits.

Phase 10: Testing and Quality Assurance

Objective	Ensure application stability, security, and correct business logic.
Tasks	**TASK-1001:** Backend unit/integration tests (Jest) for core APIs. **TASK-1002:** Verify JWT expiration and unauthorized route access blocks. **TASK-1003:** E2E walkthrough of the Event Registration journey. **TASK-1004:** E2E walkthrough of Event Creation (Admin).
Dependencies	Phases 1-9
Deliverables	Test suites, QA sign-off document.
Acceptance Criteria	100% pass rate for auth tests. Critical flows function without console errors.
Testing	Regression testing across supported browsers (Chrome, Safari, Edge).

Phase 11: Deployment

Objective	Transition application from local environment to live production.
Tasks	**TASK-1101:** Provision Production MongoDB Atlas cluster and set IP whitelists. **TASK-1102:** Deploy Backend and configure environment variables. **TASK-1103:** Deploy Frontend and configure production API URL. **TASK-1104:** Finalize CORS settings to match production frontend domain. **TASK-1105:** Verify HTTPS routing and execute a live smoke test.
Dependencies	Phase 10
Deliverables	Live URLs for Frontend and Backend API.
Acceptance Criteria	Both apps build successfully. API communicates securely over HTTPS.

Phase 12: Final Handover

Objective	Deliver documentation and operational ownership to MBU stakeholders.
Tasks	**TASK-1201:** Write detailed `README.md` with local setup commands. **TASK-1202:** Document Administrator provisioning process. **TASK-1203:** Compile troubleshooting and rollback guide. **TASK-1204:** Document known limitations and future feature pipeline.
Dependencies	Phase 11
Deliverables	Comprehensive repository documentation.
Acceptance Criteria	A new developer can clone and run the project using only the README.

4. Project Management Artifacts

A. Dependency Matrix

Feature	Relies On	Blocks
Auth & Security (Ph 3)	DB Setup (Ph 2)	Admin CRUD (Ph 5, 6, 7)
UI Design System (Ph 4)	Project Setup (Ph 1)	All Public Views (Ph 5, 6, 7, 8)
Event Management (Ph 5)	Auth (Ph 3)	Event Galleries (Ph 6)
Cloudinary Service (Ph 5)	DB Setup (Ph 2)	Event Banners (Ph 5), Galleries (Ph 6)

B. Traceability Matrix

PRD Requirement ID	Implementation Task	Test Strategy
FR-001 (Home)	TASK-404	Visual regression, Responsive check across viewports.
FR-002 (Events)	TASK-501, TASK-504	API Unit tests, UI E2E, Date-sorting logic test.
FR-003 (Registration)	TASK-505	Validation of Google Forms URL rendering and status badge logic.
FR-004 (Feedback)	TASK-801, TASK-803	API Rate limit trigger test, Input sanitization check.
FR-005 (Team)	TASK-701, TASK-704	Category relationship constraint test.
FR-006 (Admin)	TASK-301, TASK-503	JWT spoofing attempt, expired cookie handling test.

C. Deployment & Final Acceptance Checklist

- MongoDB Atlas network access restricted to Backend hosting IPs (or securely authenticated).
- Cloudinary environment variables (`CLOUD_NAME` , `API_KEY` , `API_SECRET`) securely set in backend.
- Frontend `VITE_API_URL` points to live backend domain, not `localhost` .
- Backend CORS configured to accept requests ONLY from the exact frontend domain.
- Cookie domain and `secure: true` flags set correctly for HTTPS production.
- Default administrator account bootstrapped; default password changed immediately.
- No `.env` files or hardcoded secrets found in the Git history.
- All forms (Login, Feedback) protected against basic injection and rapid re-submission.

D. Post-Launch Maintenance Plan

1. **Weekly:** Review Feedback dashboard to address student queries/issues.
2. **Monthly:** Monitor Cloudinary bandwidth/storage and MongoDB Atlas free-tier limits.
3. **Semester End:** Archive completed events using the Admin dashboard. Update the Core Team categories to reflect the graduating class and new leads. Execute a dependency update (`npm audit fix`) on the codebase.